SEN4121 – Large System Environment

Week 2: Database Design & Management

SmartStock — Supermarket Inventory, Sales Prediction & Payroll Management System

Prepared for: **ICT University Yaóundé**

Examiner: Engr. Tanwi Nkiamboh

Document version:	1.0
Group number:	Group 2
Date:	July 26, 2026
Database engine (local dev):	SQLite 3

Group Member	Matricule	Role / Section
Kwete Rayan	ICTU20241377	
Kamdeu Yamdjeuson Neil Marshall	ICTU20241386	
	ICTU2024	
	ICTU2024	

This document covers Week 2 of the SEN4121 practical exam: the Entity-Relationship design and relational schema for SmartStock's three functional modules – Inventory, Sales, and Staff & Payroll – together with indexing strategy and the backup/restore procedure used to satisfy the Week 2 build tasks.

Contents

Chapter 1
Introduction

1.1 Purpose

This document satisfies Week 2 of the SEN4121 practical exam (*Database Design & Management*, 5 marks). It presents one Entity-Relationship Diagram covering all three functional modules of SmartStock, the corresponding relational schema, the normalization rationale, the indexing strategy, and the backup/restore procedure used to demonstrate data protection.

1.2 Scope

SmartStock is organised into three modules, which play the same structural role as the exam template's Academic / Finance / HR split:

- **Inventory** – categories, suppliers, products, and stock movement history.
- **Sales** – point-of-sale transactions, line items, mobile money payments, and demand forecasts.
- **Staff & Payroll** – staff identities (shared with the Week 1 authentication service), payroll runs, and individual payslips.

1.3 Local Development Database

For local development and grading, the schema targets **SQLite 3** (a single portable file, no server process required). The schema uses only SQLite-compatible types and `CHECK` constraints in place of native `ENUM` types, so it can be ported to MySQL or PostgreSQL later by mapping `INTEGER PRIMARY KEY AUTOINCREMENT` to `SERIAL`/`AUTO_INCREMENT` and `TEXT` timestamps to `DATETIME`/`TIMESTAMP`, without changing the logical design.

Chapter 2

Entity-Relationship Diagram

2.1 Overview

Figure 2.1 shows a single combined ER diagram for all ten tables across the three modules. The diagram was authored in PlantUML (source file `er_diagram.puml`, included alongside this document) and rendered to the image below.

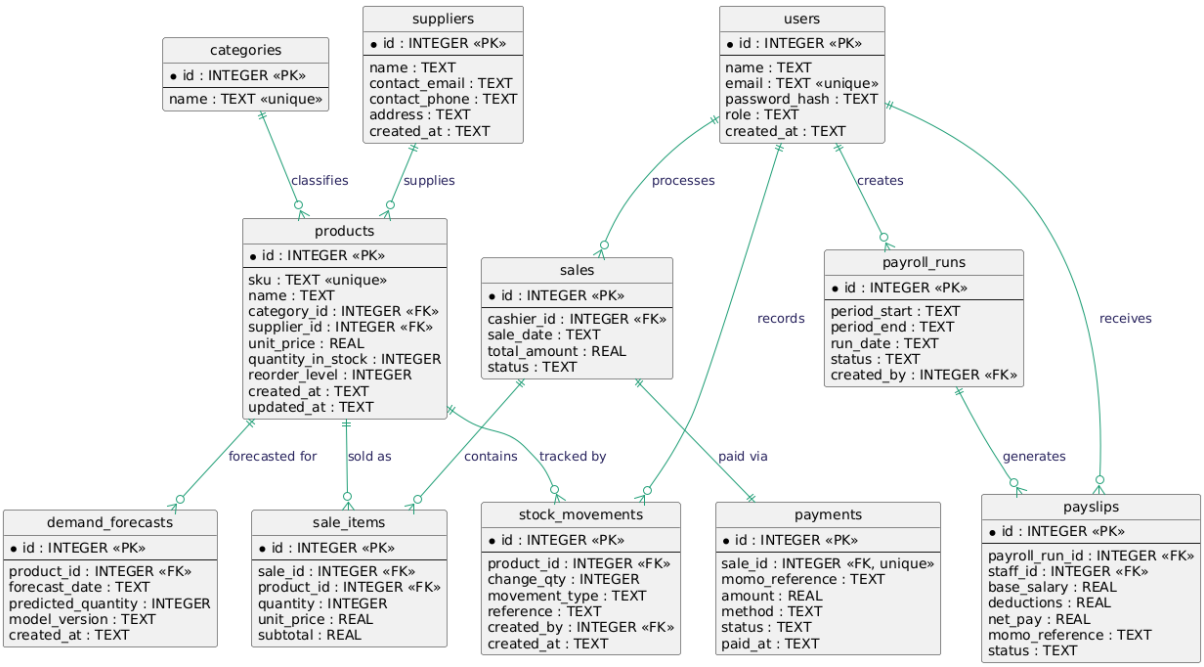

Figure 2.1: Combined Entity-Relationship Diagram for SmartStock (Inventory, Sales, Staff & Payroll)

2.2 Relationship Summary

From	Card.	To	Meaning
categories	1–N	products	A category classifies many products.
suppliers	1–N	products	A supplier supplies many products.
products	1–N	stock_movements	A product has many stock movements over time.
users	1–N	stock_movements	A staff member records many stock movements.
users	1–N	sales	A cashier processes many sales.
sales	1–N	sale_items	A sale contains many line items.
products	1–N	sale_items	A product appears in many sale line items.
sales	1–1	payments	Each sale has exactly one payment record.
products	1–N	demand_forecasts	A product has many forecast entries over time.
users	1–N	payroll_runs	An admin creates many payroll runs.
payroll_runs	1–N	payslips	A payroll run generates many payslips.
users	1–N	payslips	A staff member receives many payslips (one per run).

Chapter 3
Relational Schema

3.1 Overview

Each table below is presented with its columns, data types, and constraints. The full executable schema is provided as a single SQL file, `database/schema.sql`, so this section mirrors that file rather than duplicating a separate design. All tables are in **Third Normal Form (3NF)**: every non-key column depends only on the whole primary key, and no data that has one true source (e.g. a product's current price) is duplicated elsewhere as a mutable copy.

3.2 Module 1: Inventory

categories

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL, UNIQUE

suppliers

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL
contact_email	TEXT	
contact_phone	TEXT	
address	TEXT	
created_at	TEXT	NOT NULL, DEFAULT datetime('now')

products

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
sku	TEXT	NOT NULL, UNIQUE
name	TEXT	NOT NULL
category_id	INTEGER	NOT NULL, FK $\rightarrow$ categories(id)
supplier_id	INTEGER	FK $\rightarrow$ suppliers(id), nullable
unit_price	REAL	NOT NULL, CHECK ≥ 0
quantity_in_stock	INTEGER	NOT NULL, DEFAULT 0, CHECK ≥ 0
reorder_level	INTEGER	NOT NULL, DEFAULT 10
created_at	TEXT	NOT NULL, DEFAULT datetime('now')
updated_at	TEXT	NOT NULL, DEFAULT datetime('now')

stock_movements

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
product_id	INTEGER	NOT NULL, FK → products(id)
change_qty	INTEGER	NOT NULL (signed: positive = in, negative = out)
movement_type	TEXT	NOT NULL, CHECK IN ('RECEIPT','SALE','ADJUSTMENT','RETURN')
reference	TEXT	e.g. purchase order or sale ID
created_by	INTEGER	FK → users(id), nullable
created_at	TEXT	NOT NULL, DEFAULT datetime('now')

3.3 Module 2: Sales

sales

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
cashier_id	INTEGER	NOT NULL, FK → users(id)
sale_date	TEXT	NOT NULL, DEFAULT datetime('now')
total_amount	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0
status	TEXT	NOT NULL, DEFAULT 'pending', CHECK IN ('pending','paid','cancelled')

sale_items

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
sale_id	INTEGER	NOT NULL, FK → sales(id)
product_id	INTEGER	NOT NULL, FK → products(id)
quantity	INTEGER	NOT NULL, CHECK > 0
unit_price	REAL	NOT NULL, CHECK ≥ 0 (price snapshot at sale time)
subtotal	REAL	NOT NULL, CHECK ≥ 0

Why `unit_price` is repeated here. Storing the unit price on `sale_items` is *not* a 3NF violation: it is a historical fact ("this item sold for 500 XAF on this date"), not a duplicate of `products.unit_price`, which can change tomorrow. Removing it would make past receipts incorrect whenever a price changes.

payments

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
sale_id	INTEGER	NOT NULL, UNIQUE, FK → sales(id) (1:1)
momo_reference	TEXT	mobile money gateway reference
amount	REAL	NOT NULL, CHECK ≥ 0
method	TEXT	NOT NULL, DEFAULT 'momo', CHECK IN ('momo','cash','card')
status	TEXT	NOT NULL, DEFAULT 'pending', CHECK IN ('pending','approved','declined')
paid_at	TEXT	nullable

demand_forecasts

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
product_id	INTEGER	NOT NULL, FK → products(id)
forecast_date	TEXT	NOT NULL
predicted_quantity	INTEGER	NOT NULL, CHECK ≥ 0
model_version	TEXT	NOT NULL, DEFAULT 'v1'
created_at	TEXT	NOT NULL, DEFAULT datetime('now')

3.4 Module 3: Staff & Payroll

users (shared staff identity)

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL (bcrypt hash, from Week 1 auth-service)
role	TEXT	NOT NULL, CHECK IN ('admin','cashier')
created_at	TEXT	NOT NULL, DEFAULT datetime('now')

payroll_runs

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
period_start	TEXT	NOT NULL
period_end	TEXT	NOT NULL
run_date	TEXT	NOT NULL, DEFAULT datetime('now')
status	TEXT	NOT NULL, DEFAULT 'draft', CHECK IN ('draft','processed','paid')
created_by	INTEGER	FK → users(id), nullable

payslips

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
payroll_run_id	INTEGER	NOT NULL, FK → payroll_runs(id)
staff_id	INTEGER	NOT NULL, FK → users(id)
base_salary	REAL	NOT NULL, CHECK ≥ 0
deductions	REAL	NOT NULL, DEFAULT 0, CHECK ≥ 0
net_pay	REAL	NOT NULL, CHECK ≥ 0
momo_reference	TEXT	payout reference
status	TEXT	NOT NULL, DEFAULT 'pending', CHECK IN ('pending','paid')
UNIQUE(payroll_run_id, staff_id) – one payslip per staff member per run		

Chapter 4

Normalization Rationale (3NF)

- **1NF:** every column holds a single atomic value (e.g. no comma-separated lists of product IDs inside a text field).
- **2NF:** every table uses a single-column surrogate primary key (`id`), so there are no partial dependencies on part of a composite key.
- **3NF:** non-key columns depend only on the primary key. Concretely: a product's category name is not stored on `products` itself, only `category_id`, avoiding a transitive dependency through `categories.name`; a payslip's staff name is not duplicated from `users`, only `staff_id` is stored.
- Two deliberate, justified departures from a naive reading of 3NF are documented above: `sale_items.unit_price` (a price snapshot, i.e. a historical fact) and `sales.total_amount` (a cached aggregate of `sale_items.subtotal`, kept for fast reporting and updated in the same transaction as the line items).

Chapter 5
Indexing Strategy

5.1 Indexes Added

Index	Table.Column	Query it speeds up
idx_products_name	products.name	Searching a product by name at checkout
idx_products_category	products.category_id	Listing all products in a category
idx_stock_movements_product	stock_movements.product_id	Stock history for one product
idx_sales_sale_date	sales.sale_date	Sales reports over a date range
idx_sales_cashier	sales.cashier_id	A cashier's own sales history
idx_sale_items_sale	sale_items.sale_id	Line items for a given receipt
idx_demand_forecasts_product	demand_forecasts.product_id	Forecast history for one product
idx_payslips_staff	payslips.staff_id	A staff member's payslip history

Beyond the minimum. The exam asks for at least 2 indexes on frequently-searched tables; the schema adds 8, covering every foreign key that is likely to be filtered on and the two columns (`name`, `sale_date`) most likely to be searched live during the Week 2 evaluation. `users.email` and `products.sku` already get a free index from their `UNIQUE` constraints.

5.2 Proving Index Usage Live

Run `database/scripts/explain_query.sh` to print SQLite's query plan for two sample queries. A plan line reading `SEARCH products USING INDEX idx_products_name (name=?)` (rather than `SCAN products`) confirms the index was used instead of a full table scan.

Chapter 6
Backup and Restore Procedure

6.1 Approach

SQLite's own `.backup` command is used rather than a plain file copy, because it produces a consistent snapshot even if a connection is open, and `.restore` reloads it in place. Three scripts under `database/scripts/` automate this for the live evaluation:

1. `init_db.sh` – creates `smartstock.db` from `schema.sql` and loads `seed.sql`.
2. `backup_db.sh` – writes a timestamped copy to `database/backups/`.
3. `restore_db.sh` – restores from the most recent backup (or one given explicitly) and prints row counts per table as proof.

6.2 Demonstration Sequence (used in the evaluation)

1. `./database/scripts/init_db.sh` – fresh database with seed data.
2. `./database/scripts/backup_db.sh` – creates a backup file.
3. Delete data on purpose, e.g. `sqlite3 database/smartstock.db "DELETE FROM products;"`
4. `./database/scripts/restore_db.sh` – restores from the backup.
5. Confirm row counts are back to their original values (printed automatically by the script).